

PySpark and Apache Spark

Document ID	DOC006
Project	PySpark
Category	Spark
Batch	batch_2

Purpose: Original educational notes created as a searchable PDF corpus for a portfolio-scale incremental RAG pipeline. The material is designed to contain meaningful technical sections that naturally produce multiple retrieval chunks.

Spark Execution Model

Apache Spark is a distributed processing engine. A driver coordinates an application and executors perform tasks on partitions of data. Transformations build a logical plan and actions trigger execution. Understanding partitions and shuffles is important because distributed data movement is often more expensive than local computation.

Practical note. In Spark workloads, correctness comes first, but partitioning, shuffles, skew, and repeated computation should be considered when data volumes make distributed execution important.

Narrow and Wide Transformations

A narrow transformation can produce each output partition using data from a limited number of input partitions, often without a full shuffle. Examples include many map and filter operations. Wide transformations require data to move across partitions, as commonly happens with `groupBy`, `distinct`, `orderBy`, and many joins. Wide transformations create shuffle stages and can have significant performance cost.

Practical note. In Spark workloads, correctness comes first, but partitioning, shuffles, skew, and repeated computation should be considered when data volumes make distributed execution important.

DataFrames and Lazy Evaluation

PySpark DataFrames provide a structured API for distributed data processing. Operations such as `select`, `filter`, `withColumn`, and `join` describe transformations but are evaluated lazily. When an action such as `count`, `collect`, or `write` is requested, Spark optimizes the plan and executes the required stages.

Practical note. In Spark workloads, correctness comes first, but partitioning, shuffles, skew, and repeated computation should be considered when data volumes make distributed execution important.

Joins and Broadcast Joins

Large joins can cause expensive shuffles. When one side of a join is sufficiently small, Spark can broadcast that dataset to executors so the large side does not need to be shuffled in the same way. Broadcast joins are useful for fact-to-small-dimension patterns, but broadcasting an unexpectedly large dataset can create memory pressure.

Practical note. In Spark workloads, correctness comes first, but partitioning, shuffles, skew, and repeated computation should be considered when data volumes make distributed execution important.

Repartition and Coalesce

repartition can increase or decrease partition count and generally causes a shuffle, which can distribute data more evenly. coalesce is primarily used to reduce partitions with less data movement and may produce uneven partition sizes. The appropriate choice depends on whether redistribution is required and on downstream workload characteristics.

Practical note. In Spark workloads, correctness comes first, but partitioning, shuffles, skew, and repeated computation should be considered when data volumes make distributed execution important.

Caching and Persistence

Caching can help when the same expensive DataFrame is reused multiple times. persist allows explicit storage levels, while cache uses a default storage behavior. Caching everything wastes executor memory and can hurt performance, so intermediate results should be cached only when reuse justifies the cost.

Practical note. In Spark workloads, correctness comes first, but partitioning, shuffles, skew, and repeated computation should be considered when data volumes make distributed execution important.

End of document.